will consider SDPs in the following. The next theorem states that a delayed reward MDP $\tilde{\mathcal{P}}$ with a particular policy π can be transformed into a return-equivalent SDP $\mathcal{P}$ with an optimal reward redistribution.

Theorem 2. *We assume a delayed reward MDP $\tilde{\mathcal{P}}$, where the accumulated reward is given at sequence end. A new SDP $\mathcal{P}$ is obtained by a second order Markov reward redistribution, which ensures that $\mathcal{P}$ is return-equivalent to $\tilde{\mathcal{P}}$. For a specific π, the following two statements are equivalent:*
(I) $\kappa(T-t-1,t) = 0$*, i.e. the reward redistribution is optimal,*

$$\textit{(II)}\ \mathrm{E}\left[R_{t+1} \mid s_{t-1}, a_{t-1}, s_t, a_t\right] \;=\; \tilde{q}^{\pi}(s_t, a_t) \;-\; \tilde{q}^{\pi}(s_{t-1}, a_{t-1})\,. \tag{1}$$

An optimal reward redistribution fulfills for $1 \leqslant t \leqslant T$ and $0 \leqslant m \leqslant T-t$: $\kappa(m, t-1) = 0$.

The proof can be found after Theorem A4 in the appendix. Equation $\kappa(T-t-1,t) = 0$ implies that the new SDP $\mathcal{P}$ has no delayed rewards, that is, $\mathrm{E}_{\pi}\left[R_{t+1+\tau} \mid s_{t-1}, a_{t-1}\right] = 0$, for $0 \leqslant \tau \leqslant T-t-1$ (Corollary A1 in the appendix). The SDP $\mathcal{P}$ has no delayed rewards since no state-action pair can increase or decrease the expectation of a future reward. Equation (1) shows that for an optimal reward redistribution the expected reward has to be the difference of consecutive Q-values of the original delayed reward. The optimal reward redistribution is second order Markov since the expectation of R_{t+1} at time $(t+1)$ depends on $(s_{t-1}, a_{t-1}, s_t, a_t)$.

The next theorem states the major advantage of an optimal reward redistribution: $\tilde{q}^{\pi}(s_t, a_t)$ can be estimated with an offset that depends only on s_t by estimating the expected immediate redistributed reward. Thus, Q-value estimation becomes trivial and the the advantage function of the MDP $\tilde{\mathcal{P}}$ can be readily computed.

Theorem 3. *If the reward redistribution is optimal, then the Q-values of the SDP $\mathcal{P}$ are given by*

$$q^{\pi}(s_t, a_t) \;=\; r(s_t, a_t) \;=\; \tilde{q}^{\pi}(s_t, a_t) \;-\; \mathrm{E}_{s_{t-1}, a_{t-1}}\left[\tilde{q}^{\pi}(s_{t-1}, a_{t-1}) \mid s_t\right] \;=\; \tilde{q}^{\pi}(s_t, a_t) \;-\; \psi^{\pi}(s_t)\,. \tag{2}$$

The SDP $\mathcal{P}$ and the original MDP $\tilde{\mathcal{P}}$ have the same advantage function. Using a behavior policy $\breve{\pi}$ the expected immediate reward is

$$\mathrm{E}_{\breve{\pi}}\left[R_{t+1} \mid s_t, a_t\right] \;=\; \tilde{q}^{\pi}(s_t, a_t) \;-\; \psi^{\pi, \breve{\pi}}(s_t)\,. \tag{3}$$

The proof can be found after Theorem A5 in the appendix. If the reward redistribution is not optimal, then $\kappa(T-t-1,t)$ measures the deviation of the Q-value from $r(s_t, a_t)$. This theorem justifies several learning methods based on reward redistribution presented in the next paragraph.

Novel Learning Algorithms Based on Reward Redistributions. We assume $\gamma = 1$ and a finite horizon or an absorbing state original MDP $\tilde{\mathcal{P}}$ with delayed rewards. For this setting we introduce new reinforcement learning algorithms. They are gradually changing the reward redistribution during learning and are based on the estimations in Theorem 3. These algorithms are also valid for non-optimal reward redistributions, since the optimal policies are kept (Theorem 1). Convergence of RUDDER learning can under standard assumptions be proven by the stochastic approximation for two time-scale update rules [17, 64]. Learning consists of an LSTM and a Q-value update. Convergence proofs to an optimal policy are difficult, since locally stable attractors may not correspond to optimal policies.

According to Theorem 1, reward redistributions keep the optimal policies. Therefore, even non-optimal reward redistributions ensure correct learning. However, an optimal reward redistribution speeds up learning considerably. Reward redistributions can be combined with methods that use Q-value ranks or advantage functions. We consider (A) Q-value estimation, (B) policy gradients, and (C) Q-learning. Type (A) methods estimate Q-values and are divided into variants (i), (ii), and (iii). Variant (i) assumes an optimal reward redistribution and estimates $\tilde{q}^{\pi}(s_t, a_t)$ with an offset depending only on s_t. The estimates are based on Theorem 3 either by on-policy direct Q-value estimation according to Eq. (2) or by off-policy immediate reward estimation according to Eq. (3). Variant (ii) methods assume a non-optimal reward redistribution and correct Eq. (2) by estimating κ. Variant (iii) methods use eligibility traces for the redistributed reward. RUDDER learning can be based on policies like "greedy in the limit with infinite exploration" (GLIE) or "restricted rank-based randomized" (RRR) [118]. GLIE policies change toward greediness with respect to the Q-values during learning. For more details on these learning approaches see Section A2.7.1 in the apendix.

Type (B) methods replace in the expected updates $\mathrm{E}_{\pi}\left[\nabla_{\theta} \log \pi(a \mid s; \boldsymbol{\theta}) q^{\pi}(s, a)\right]$ of policy gradients the value $q^{\pi}(s, a)$ by an estimate of $r(s, a)$ or by a sample of the redistributed reward. The offset

$\psi^\pi(s)$ in Eq. (2) or $\psi^{\pi,\tilde{\pi}}(s)$ in Eq. (3) reduces the variance as baseline normalization does. These methods can be extended to Trust Region Policy Optimization (TRPO) [113] as used in Proximal Policy Optimization (PPO) [115]. The type (C) method is Q-learning with the redistributed reward. Here, Q-learning is justified if immediate and future reward are drawn together, as typically done.

3 Constructing Reward Redistributions by Return Decomposition

We now propose methods to construct reward redistributions. Learning with non-optimal reward redistributions *does work* since the optimal policies do not change according to Theorem 1. However, reward redistributions that are optimal considerably speed up learning, since future expected rewards introduce biases in TD methods and high variances in MC methods. The expected optimal redistributed reward is the difference of Q-values according to Eq. (1). The more a reward redistribution deviates from these differences, the larger are the absolute κ-values and, in turn, the less optimal the reward redistribution gets. Consequently, to construct a reward redistribution which is close to optimal we aim at identifying the largest Q-value differences.

Reinforcement Learning as Pattern Recognition. We want to transform the reinforcement learning problem into a pattern recognition task to employ deep learning approaches. The sum of the Q-value differences gives the difference between expected return at sequence begin and the expected return at sequence end (telescope sum). Thus, Q-value differences allow to predict the expected return of the whole state-action sequence. Identifying the largest Q-value differences reduces the prediction error most. Q-value differences are assumed to be associated with patterns in state-action transitions. The largest Q-value differences are expected to be found more frequently in sequences with very large or very low return. The resulting task is to predict the expected return from the whole sequence and identify which state-action transitions have contributed the most to the prediction. This pattern recognition task serves to construct a reward redistribution, where the redistributed reward corresponds to the different contributions. The next paragraph shows how the return is decomposed and redistributed along the state-action sequence.

Return Decomposition. The *return decomposition* idea is that a function g predicts the expectation of the return for a given state-action sequence (return for the whole sequence). The function g is neither a value nor an action-value function since it predicts the expected return when the whole sequence is given. With the help of g either the predicted value or the realization of the return is redistributed over the sequence. A state-action pair receives as redistributed reward its contribution to the prediction, which is determined by contribution analysis. We use contribution analysis since sensitivity analysis has serious drawbacks: local minima, instabilities, exploding or vanishing gradients, and proper exploration [48, 110]. The major drawback is that the relevance of actions is missed since sensitivity analysis does not consider the contribution of actions to the output, but only their effect on the output when slightly perturbing them. Contribution analysis determines how much a state-action pair contributes to the final prediction. We can use any contribution analysis method, but we specifically consider three methods: (A) differences of return predictions, (B) integrated gradients (IG) [125], and (C) layer-wise relevance propagation (LRP) [3]. For (A), g must try to predict the sequence-wide return at every time step. The redistributed reward is given by the difference of consecutive predictions. The function g can be decomposed into past, immediate, and future contributions to the return. Consecutive predictions share the same past and the same future contributions except for two immediate state-action pairs. Thus, in the difference of consecutive predictions contributions cancel except for the two immediate state-action pairs. Even for imprecise predictions of future contributions to the return, contribution analysis is more precise, since prediction errors cancel out. Methods (B) and (C) rely on information later in the sequence for determining the contribution and thereby may introduce a non-Markov reward. The reward can be viewed to be probabilistic but is prone to have high variance. Therefore, we prefer method (A).

Explaining Away Problem. We still have to tackle the problem that reward causing actions do not receive redistributed rewards since they are explained away by later states. To describe the problem, assume an MDP $\tilde{\mathcal{P}}$ with the only reward at sequence end. To ensure the Markov property, states in $\tilde{\mathcal{P}}$ have to store the reward contributions of previous state-actions; e.g. s_T has to store all previous contributions such that the expectation $\tilde{r}(s_T, a_T)$ is Markov. The explaining away problem is that later states are used for return prediction, while reward causing earlier actions are missed.

To avoid explaining away, we define a difference function $\Delta(s_{t-1}, a_{t-1}, s_t, a_t)$ between a state-action pair (s_t, a_t) and its predecessor (s_{t-1}, a_{t-1}). That Δ is a function of $(s_t, a_t, s_{t-1}, a_{t-1})$ is justified by Eq. (1), which ensures that such Δs allow an optimal reward redistribution. The sequence of differences is $\Delta_{0:T} := \big(\Delta(s_{-1}, a_{-1}, s_0, a_0), \ldots, \Delta(s_{T-1}, a_{T-1}, s_T, a_T)\big)$. The components Δ are assumed to be statistically independent from each other, therefore Δ cannot store reward contributions of previous Δ. The function g should predict the return by $g(\Delta_{0:T}) = \tilde{r}(s_T, a_T)$ and can be decomposed into $g(\Delta_{0:T}) = \sum_{t=0}^{T} h_t$. The contributions are $h_t = h(\Delta(s_{t-1}, a_{t-1}, s_t, a_t))$ for $0 \leqslant t \leqslant T$. For the redistributed rewards R_{t+1}, we ensure $\mathrm{E}\left[R_{t+1} \mid s_{t-1}, a_{t-1}, s_t, a_t\right] = h_t$. The reward $\tilde{R}_{T+1}$ of $\tilde{\mathcal{P}}$ is probabilistic and the function g might not be perfect, therefore neither $g(\Delta_{0:T}) = \tilde{r}_{T+1}$ for the return realization $\tilde{r}_{T+1}$ nor $g(\Delta_{0:T}) = \tilde{r}(s_T, a_T)$ for the expected return holds. Therefore, we need to introduce the compensation $\tilde{r}_{T+1} - \sum_{\tau=0}^{T} h(\Delta(s_{\tau-1}, a_{\tau-1}, s_\tau, a_\tau))$ as an extra reward R_{T+2} at time $T+2$ to ensure strictly return-equivalent SDPs. If g was perfect, then it would predict the expected return which could be redistributed. The new redistributed rewards R_{t+1} are based on the return decomposition, since they must have the contributions h_t as mean:
$\mathrm{E}\left[R_1 \mid s_0, a_0\right] = h_0$, $\mathrm{E}\left[R_{t+1} \mid s_{t-1}, a_{t-1}, s_t, a_t\right] = h_t, 0 < t \leqslant T$, $R_{T+2} = \tilde{R}_{T+1} - \sum_{t=0}^{T} h_t$, where the realization $\tilde{r}_{T+1}$ is replaced by its random variable $\tilde{R}_{T+1}$. If the prediction of g is perfect, then we can redistribute the expected return via the prediction. Theorem 2 holds also for the correction R_{T+2} (see Theorem A6 in the appendix). A g with zero prediction errors results in an optimal reward redistribution. Small prediction errors lead to reward redistributions close to an optimal one.

RUDDER: Return Decomposition using LSTM. RUDDER uses a Long Short-Term Memory (LSTM) network for return decomposition and the resulting reward redistribution. RUDDER consists of three phases. **(I) Safe exploration.** Exploration sequences should generate LSTM training samples with delayed rewards by avoiding low Q-values during a particular time interval. Low Q-values hint at states where the agent gets stuck. Parameters comprise starting time, length, and Q-value threshold. **(II) Lessons replay buffer for training the LSTM.** If RUDDER's safe exploration discovers an episode with unseen delayed rewards, it is secured in a lessons replay buffer [74]. Unexpected rewards are indicated by a large prediction error of the LSTM. For LSTM training, episodes with larger errors are sampled more often from the buffer, similar to prioritized experience replay [109]. **(III) LSTM and return decomposition.** An LSTM learns to predict sequence-wide return at every time step and, thereafter, return decomposition uses differences of return predictions (contribution analysis method (A)) to construct a reward redistribution. For more details see Section A8.4 in the appendix.

Feedforward Neural Networks (FFNs) vs. LSTMs. In contrast to LSTMs, FNNs are not suited for processing sequences. Nevertheless, FNNs can learn a action-value function, which enables contribution analysis by differences of predictions. However, this leads to serious problems by spurious contributions that hinder learning. For example, any contributions would be incorrect if the true expectation of the return did not change. Therefore, prediction errors might falsely cause contributions leading to spurious rewards. FNNs are prone to such prediction errors since they have to predict the expected return again and again from each different state-action pair and cannot use stored information. In contrast, the LSTM is less prone to produce spurious rewards: (i) The LSTM will only learn to store information if a state-action pair has a strong evidence for a change in the expected return. If information is stored, then internal states and, therefore, also the predictions change, otherwise the predictions stay unchanged. Hence, storing events receives a contribution and a corresponding reward, while by default nothing is stored and no contribution is given. (ii) The LSTM tends to have smaller prediction errors since it can reuse past information for predicting the expected return. For example, key events can be stored. (iii) Prediction errors of LSTMs are much more likely to cancel via prediction differences than those of FNNs. Since consecutive predictions of LSTMs rely on the same internal states, they usually have highly correlated errors.

Human Expert Episodes. They are an alternative to exploration and can serve to fill the lessons replay buffer. Learning can be sped up considerably when LSTM identifies human key actions. Return decomposition will reward human key actions even for episodes with low return since other actions that thwart high returns receive negative reward. Using human demonstrations in reinforcement learning led to a huge improvement on some Atari games like Montezuma's Revenge [93, 2].